SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA1214	**Course Title:**	Computer Architecture
CO Assessed:	CO3 – Precision (BL3)	**Assessment:**	Experiments
Weightage:	45%	**Total Marks:**	100
CO3 Statement:	Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT).		
Student Name:	S.Lakshmi Priya	**Reg. No.:**	192524284
Section / Batch:	B.Tech AI/DS	**Date:**	11/08/26

Instructions to Students

- Identify the relevant concepts of register transfers, ALU operations, bus organization, pipelining, hazards, and superscalar processing.
- Analyze the execution of data transfers, arithmetic and logic operations, instruction processing, and parallel execution.
- Apply suitable techniques such as multiple buses, pipelining, stalling, forwarding, branch prediction, and speculative execution.
- Compute performance measures such as execution time, clock cycles, speedup, throughput, and resource utilization.
- Interpret the results by evaluating performance improvements, bottlenecks, and the suitability of each architectural technique.

QUESTIONS

1. Analyze a processor datapath that performs register transfers and logical operations such as XOR, NOT, and comparison, and examine the control signals and execution sequence required for each operation.

2. Develop a bus-based system for transferring data between the processor, memory, and I/O devices, and analyze how bus width, transfer rate, and bus contention influence system performance.

3. Simulate a pipelined processor with arithmetic, memory, and branch instructions, and evaluate the effects of pipeline depth on execution time, throughput, and overall speedup.

4. Construct a pipeline containing instruction dependencies and branch conditions, and analyze the performance impact of data and control hazards using stalling, forwarding, and branch prediction techniques.

5. Develop a superscalar processor model that supports multiple execution units and out-of-order instruction processing, and evaluate how instruction scheduling and resource allocation improve instruction throughput.

Code of Conduct

I S.Lakshmi Priya (192524284)__(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S. Lakshmi Priya _____

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Q. No	Understanding of Concepts(25)	Experimental Design (30)	Use of Tools(20)	Analysis of Results (15)	Documentation (10)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	**/ 30**	**/ 20**	**/ 15**	**/ 10**	**/ 100**

Course Coordinator **Faculty Incharge**

QUESTION 1

Analysis of a Processor Datapath for Register Transfers and Logical Operations

1. Introduction

A processor datapath is the part of a CPU that performs data processing and transfers data between different internal components. It consists of registers, an Arithmetic Logic Unit (ALU), multiplexers, buses, and control circuitry. The datapath is responsible for carrying out operations specified by machine instructions.

Register transfer operations move data from one register to another, while logical operations such as XOR, NOT, and comparison process data according to the instruction. The control unit generates control signals that determine which registers are selected, what operation the ALU performs, and where the resulting data is stored.

The execution of an instruction generally follows a sequence of steps: fetch, decode, execute, and write-back. During these steps, the required control signals are activated in the correct order so that data moves through the datapath correctly.

2. Basic Components of the Processor Datapath

The main components involved in register transfers and logical operations are:

a) Registers

Registers are small, high-speed storage locations inside the processor. They temporarily store operands, addresses, and intermediate results. Examples include general-purpose registers such as R1, R2, and R3.

b) ALU

The Arithmetic Logic Unit (ALU) performs arithmetic and logical operations. For this analysis, the important logical operations are XOR, NOT, and comparison.

c) Multiplexer

A multiplexer selects one input from several possible inputs and sends it to the required datapath component. It helps determine which register or data source should be connected to the ALU.

d) Internal Bus

A bus provides a path for transferring data between registers, the ALU, memory interface, and other processor components.

e) Control Unit

The control unit generates control signals based on the decoded instruction. These signals control register selection, ALU operation, data movement, and writing of results.

3. Register Transfer Operation

A register transfer moves data from one register to another.

For example:

$R1 \leftarrow R2$

This means the contents of register R2 are copied into register R1. The original contents of R2 remain unchanged.

The execution sequence is:

1. The instruction is fetched from memory.

2. The control unit decodes the instruction.
3. The source register R2 is selected.
4. The contents of R2 are placed on the internal bus.
5. The destination register R1 is enabled.
6. On the active clock edge, R1 loads the data from the bus.

The important control signals may include:

- R2out – enables R2 to place its contents on the bus.
- R1in – enables R1 to receive data.
- Clock – controls when the transfer takes place.

Thus, the transfer can be represented as:

R2 → Bus → R1

This operation demonstrates how the control unit coordinates data movement within the processor.

4. XOR Operation

The XOR (Exclusive OR) operation compares two binary operands bit by bit. The output is 1 when the corresponding input bits are different and 0 when they are the same.

For example:

R3 ← R1 XOR R2

The execution sequence is:

1. The instruction is fetched and decoded.
2. R1 and R2 are selected as source operands.
3. The operands are supplied to the ALU.
4. The control unit sends the XOR control signal to the ALU.
5. The ALU performs the XOR operation.
6. The result is placed on the datapath.
7. The destination register R3 is enabled.
8. R3 stores the result on the clock edge.

The simplified datapath is:

R1 + R2 → ALU (XOR) → R3

The main control signals are R1out, R2out, ALU_XOR, and R3in. In an actual processor, multiplexers may be used to select the two ALU input operands rather than using a single common bus.

5. NOT Operation

The NOT operation performs a bitwise inversion. Every 0 becomes 1, and every 1 becomes 0.

For example:

R2 ← NOT R1

If R1 contains:

10110010

the result is:

01001101

The execution sequence is:

1. The instruction is fetched from memory.
2. The control unit decodes the instruction.
3. R1 is selected as the source register.
4. R1 supplies its contents to the ALU.

5. The control unit selects the NOT operation.
6. The ALU inverts every bit of the operand.
7. The result is sent to the destination register R2.
8. R2 stores the result at the active clock edge.

The datapath can be represented as:
R1 → ALU (NOT) → R2
The main control signals are R1out, ALU_NOT, and R2in.

6. Comparison Operation

A comparison operation determines the relationship between two operands. It can determine whether two values are equal, or whether one value is greater than or less than another.
For example:
Compare R1, R2
The processor sends both operands to the ALU or comparison circuitry. The result may be represented using status flags such as:

- Zero (Z) – indicates that the operands are equal when the comparison result is zero.
- Carry (C) – may provide information about unsigned comparisons.
- Negative/Sign (N/S) – indicates the sign of a result.
- Overflow (V) – indicates signed arithmetic overflow where applicable.

The execution sequence is:

1. The comparison instruction is fetched.
2. The control unit decodes it.
3. R1 and R2 are selected as operands.
4. The operands are supplied to the ALU.
5. The control unit selects the comparison operation.
6. The ALU compares the operands, commonly by performing a subtraction internally.
7. The appropriate status flags are updated.
8. A conditional branch instruction can use these flags to determine the next instruction.

For example, if R1 = R2, the zero flag may be set to 1, indicating equality.

7. Control Signals and Execution Sequence

Control signals are essential because they coordinate all datapath activities. Each operation requires a specific combination of signals.

Operation	Source	ALU/Control Function	Destination/Result
Register Transfer	R2	Transfer	R1
XOR	R1, R2	XOR	R3
NOT	R1	NOT	R2
Comparison	R1, R2	Compare/Subtract	Status Flags

The control unit first determines the operation from the instruction opcode. It then activates the required signals in the correct sequence. The signals ensure that the correct source registers are selected, the ALU performs the required function, and the result is stored or used appropriately.

8. Overall Execution Sequence

The general instruction execution process can be summarized as:
Fetch → Decode → Select Operands → Execute → Store Result/Update Flags
During the fetch stage, the processor obtains the instruction from memory. During decode, the control unit determines the required operation and operands. During execute, data moves

through the datapath and the ALU performs the required operation. Finally, during write-back, the result is stored in the destination register or status flags are updated.
The clock signal synchronizes these activities so that each operation occurs at the correct time.

9. Conclusion

A processor datapath provides the hardware structure required for transferring and processing data inside the CPU. Register transfers move data between registers, while logical operations such as XOR and NOT are performed by the ALU. Comparison operations evaluate operands and update status flags that can influence later instructions.
The control unit plays a central role by generating appropriate control signals for each stage of execution. By carefully coordinating source selection, ALU functions, destination registers, and clock signals, the processor can execute instructions accurately and efficiently. Therefore, understanding the datapath and its control signals is essential for understanding how a processor executes machine-level instructions.

QUESTION – 2

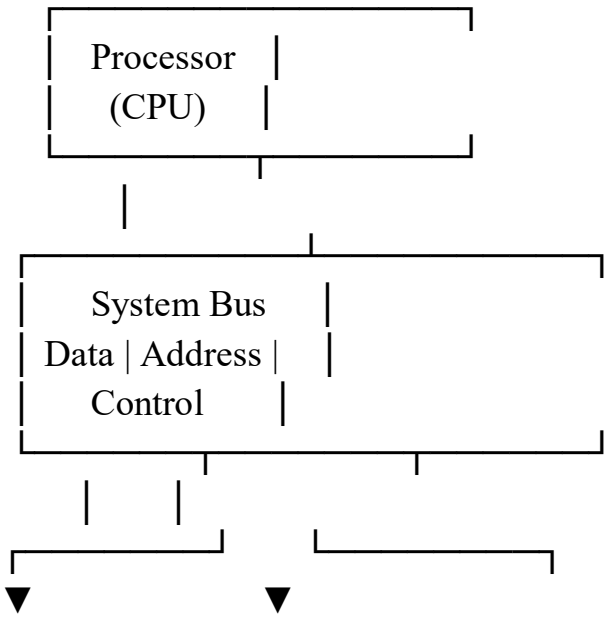
Bus-Based System for Data Transfer Between Processor, Memory, and I/O Devices

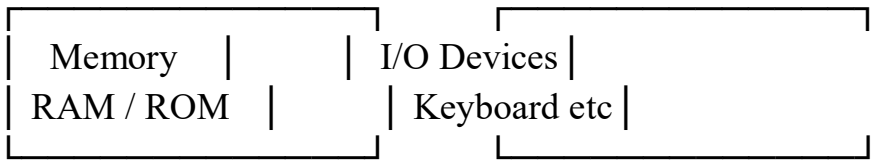
1. Introduction

A bus-based system is a computer architecture in which the processor, memory, and input/output (I/O) devices communicate through a common set of communication lines called a bus. The bus provides a path for transferring data, addresses, and control information between different components of the computer system.
A typical bus-based system contains three major buses: the data bus, address bus, and control bus. The performance of the system depends on several factors, particularly bus width, transfer rate, and bus contention. Proper design of these factors allows efficient communication between the CPU, memory, and I/O devices.

2. Basic Structure of a Bus-Based System

A simple bus-based system can be represented as:





The processor communicates with memory and I/O devices by placing appropriate addresses, data, and control signals on the system bus.

3. Types of Buses

a) Data Bus

The data bus carries actual data between the processor, memory, and I/O devices. It is generally bidirectional because data may travel from the CPU to memory or from memory to the CPU.

For example, during a memory read operation, memory places the requested data on the data bus, and the processor receives it.

b) Address Bus

The address bus carries the address of the memory location or I/O device that the processor wants to access. It is generally unidirectional from the processor to the other components.

For example, when the CPU wants to read data from a particular memory location, it places that location's address on the address bus.

c) Control Bus

The control bus carries control and timing signals that coordinate data transfers. Important signals include:

- Read – indicates that data should be read.
- Write – indicates that data should be written.
- Clock – synchronizes operations.
- Interrupt – allows an I/O device to request CPU attention.
- Bus Request/Grant – controls access to the bus.

4. Data Transfer Between Processor and Memory

Consider a memory read operation. The processor first places the required memory address on the address bus. It then activates the Read control signal. Memory recognizes the address and places the requested data on the data bus. Finally, the processor receives and stores the data.

The sequence is:

CPU → Address Bus → Memory

Memory → Data Bus → CPU

For a memory write operation, the CPU places the address and data on the respective buses and activates the Write signal. Memory then stores the data at the specified address.

5. Data Transfer Between Processor and I/O Devices

The same bus can be used to communicate with I/O devices such as keyboards, displays, printers, storage devices, and network interfaces.

For example, when the processor reads data from a keyboard interface:

1. The CPU selects the I/O device.
2. The device address is placed on the address bus.
3. The CPU sends a read control signal.
4. The I/O controller places the input data on the data bus.
5. The processor receives the data.

This allows several I/O devices to communicate with the processor using a common communication structure.

6. Effect of Bus Width on Performance

Bus width refers to the number of bits that can be transferred simultaneously through the data bus.

For example:

- An 8-bit bus can transfer 8 bits at a time.
- A 16-bit bus can transfer 16 bits at a time.
- A 32-bit bus can transfer 32 bits at a time.
- A 64-bit bus can transfer 64 bits at a time.

A wider bus can transfer more data during each bus cycle. Therefore, increasing the bus width generally improves data-transfer performance.

For example, a 64-bit bus can transfer twice as much data per transfer as a 32-bit bus, assuming the same bus frequency and conditions. However, a wider bus also requires more physical connections and can increase hardware complexity and cost.

7. Effect of Transfer Rate on Performance

The bus transfer rate indicates how quickly data can be transferred over the bus. It depends on factors such as bus frequency, number of transfers per clock cycle, and bus width.

A simplified relationship is:

Data Transfer Rate = Bus Width × Number of Transfers per Second

For example, if a bus transfers 64 bits per cycle and operates at a high clock frequency, it can provide a high data-transfer rate.

A higher transfer rate allows the processor to access memory and I/O devices more quickly. This reduces waiting time and improves overall system performance. However, the actual performance may also be limited by memory speed, device speed, processor architecture, and bus protocol.

8. Bus Contention

Bus contention occurs when two or more devices attempt to use the shared bus at the same time. Since a common bus normally cannot allow multiple devices to drive the same signals simultaneously, contention must be controlled.

For example, if the processor and an I/O controller attempt to transmit data on the bus simultaneously, their signals may interfere with each other. This can result in incorrect data and reduced system reliability.

To prevent this problem, the system uses bus arbitration. Bus arbitration determines which device is allowed to use the bus at a particular time.

Common approaches include:

- Centralized arbitration – a central bus controller decides which device gets access.
- Distributed arbitration – participating devices collectively determine bus ownership.
- Priority-based arbitration – higher-priority devices receive bus access first.

Effective arbitration reduces waiting time and prevents conflicts.

9. Influence of Bus Contention on System Performance

Bus contention can significantly reduce performance when many devices require frequent bus access. A device may have to wait until another device finishes its transfer.

High contention can cause:

- Increased waiting time.
- Reduced effective transfer rate.
- Processor delays.
- Lower I/O performance.
- Increased response time.

Therefore, a good bus-based architecture must provide efficient arbitration and scheduling mechanisms.

10. Overall Performance Analysis

The performance of a bus-based system depends on the combined effect of bus width, transfer rate, and contention.

A wider bus increases the amount of data transferred per cycle. A higher transfer rate increases the number of transfers completed in a given period. However, excessive bus contention can prevent the system from achieving the theoretical maximum transfer rate. For example, a system may have a very wide and fast bus, but if several high-speed I/O devices continuously compete for access, the effective performance can still be low.

11. Conclusion

A bus-based system provides an efficient communication mechanism between the processor, memory, and I/O devices. The data bus carries data, the address bus identifies the destination or source, and the control bus manages the timing and type of transfer.

Bus width determines how much data can be transferred at once, while transfer rate determines how quickly transfers occur. Bus contention occurs when multiple devices compete for the shared bus and can reduce system performance. Therefore, a well-designed bus system requires an appropriate bus width, high transfer rate, and effective bus arbitration mechanism to achieve reliable and efficient computer-system performance**.**

QUESTION -3

Simulation and Evaluation of a Pipelined Processor

1. Introduction

A pipelined processor is a processor architecture in which the execution of instructions is divided into several stages. Different instructions can occupy different stages at the same time, similar to an assembly line. This improves the throughput of the processor because multiple instructions are processed simultaneously.

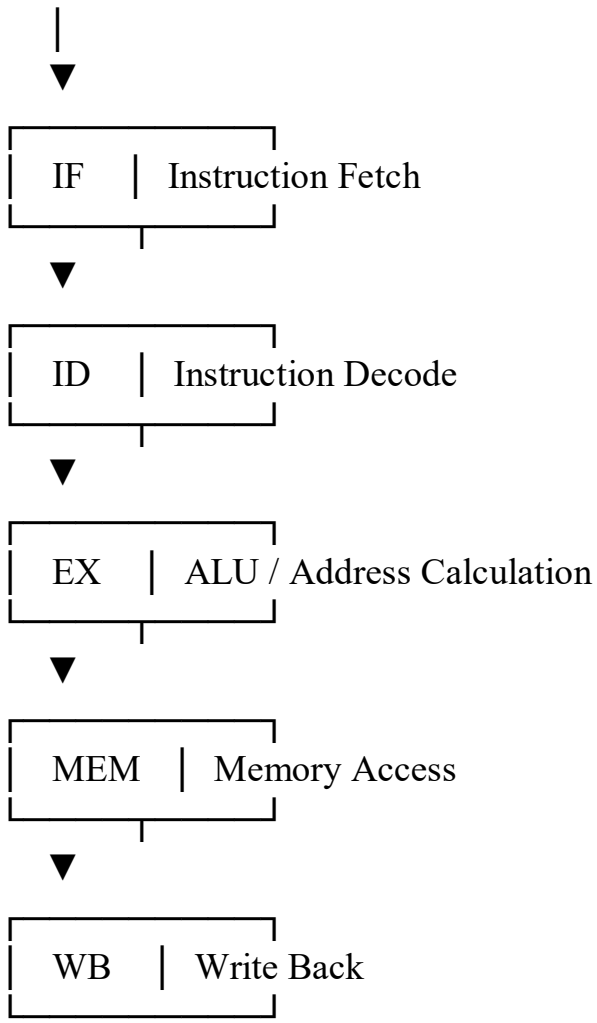
A typical five-stage pipeline consists of:

1. IF – Instruction Fetch
2. ID – Instruction Decode
3. EX – Execute
4. MEM – Memory Access
5. WB – Write Back

The pipeline can execute arithmetic, memory, and branch instructions by passing them through these stages in sequence.

2. Pipeline Structure

The basic structure of a five-stage pipelined processor is:



Each stage performs a specific task. Pipeline registers are placed between stages to store intermediate results and control information.

3. Instructions Used in the Simulation

To evaluate the pipeline, three categories of instructions can be considered.

a) Arithmetic Instruction

Example:

ADD R1, R2, R3

This instruction adds the contents of R2 and R3 and stores the result in R1.

Execution:

IF → ID → EX → MEM → WB

The main operation occurs during the EX stage, where the ALU performs the addition.

b) Memory Instruction

Example:

LOAD R4, 0(R5)

This instruction calculates a memory address and loads data from memory into R4.

Execution:

IF → ID → EX → MEM → WB

The address is calculated during EX, memory is accessed during MEM, and the loaded data is written into R4 during WB.

c) Branch Instruction

Example:

BEQ R1, R2, TARGET

This instruction checks whether R1 and R2 are equal. If they are equal, execution branches to the target instruction.

The branch instruction requires comparison and target-address calculation. If the processor determines that the branch is taken after later instructions have already entered the pipeline, those instructions may have to be discarded. This is called a pipeline flush.

4. Pipeline Simulation

Consider the following instruction sequence:

I1: ADD R1, R2, R3

I2: LOAD R4, 0(R5)

I3: ADD R6, R1, R4

I4: BEQ R6, R7, TARGET

I5: SUB R8, R9, R10

A simplified five-stage pipeline schedule can be represented as:

Instruction	Cycle 1	Cycle 2	Cycle 3	Cycle 4	Cycle 5	Cycle 6	Cycle 7	Cycle 8
I1:	IF	ID	E	M	W			

Instruction	Cycle 1	Cycle 2	Cycle 3	Cycle 4	Cycle 5	Cycle 6	Cycle 7	Cycle 8
ADD			X	EM	B			
I2: LOAD		IF	ID	EX	MEM	WB		
I3: ADD			IF	ID	EX	MEM	WB	
I4: BEQ				IF	ID	EX	MEM	WB
I5: SUB					IF	ID	EX	MEM

This illustrates the main advantage of pipelining: after the pipeline is filled, several instructions are being processed during the same clock cycle.

5. Effect of Pipeline Depth

Pipeline depth refers to the number of stages into which instruction execution is divided.

For example:

- A 3-stage pipeline may divide execution into three major stages.
- A 5-stage pipeline may use IF, ID, EX, MEM, and WB.
- A deeper pipeline may divide these operations into more smaller stages.

Increasing pipeline depth can reduce the amount of work performed in each individual stage. This can allow the processor to operate at a higher clock frequency.

However, deeper pipelines also introduce additional pipeline registers, increased control complexity, and potentially greater branch penalties.

6. Effect on Execution Time

For a non-pipelined processor, instructions are generally completed one after another. If each instruction requires approximately five stages, five clock cycles may be required per instruction.

For a pipeline with k stages and n instructions, the ideal execution time is approximately:

$T = (k + n - 1) \times$ clock cycle time

For example, for a five-stage pipeline executing 10 independent instructions:

$T = 5 + 10 - 1 = 14$ cycles

Without pipelining, assuming five cycles per instruction:

$T = 10 \times 5 = 50$ cycles

Thus, pipelining can significantly reduce total execution time for a sequence of independent instructions.

7. Effect on Throughput

Throughput is the number of instructions completed per unit of time.

A non-pipelined processor may complete approximately one instruction every several cycles. In an ideal five-stage pipeline, once the pipeline is full, approximately one instruction can complete every clock cycle.

Therefore, pipeline depth primarily improves throughput, rather than reducing the fundamental work required for an individual instruction.

A deeper pipeline can also permit a shorter clock period if each stage contains less logic. This may further increase the instruction completion rate.

8. Effect on Speedup

Speedup measures how much faster the pipelined processor operates compared with a non-pipelined processor.

The ideal speedup is approximately:

Speedup = Non-pipelined execution time / Pipelined execution time

For a large number of independent instructions, a five-stage pipeline can approach a speedup close to 5 times, assuming equal stage delays and no hazards.

For a small number of instructions, the speedup is lower because time is required to fill and drain the pipeline.

9. Pipeline Hazards

The simulation must also consider pipeline hazards, which can reduce the ideal performance.

a) Data Hazard

A data hazard occurs when one instruction depends on the result of an earlier instruction.

For example:

I1: ADD R1, R2, R3

I2: SUB R4, R1, R5

I2 requires the value produced by I1. Techniques such as forwarding or pipeline stalls can be used to solve this problem.

b) Control Hazard

A control hazard occurs because of branch instructions. The processor may not know the correct next instruction until the branch condition has been evaluated.

Branch prediction, delayed branching, or pipeline flushing can be used to reduce the effect.

c) Structural Hazard

A structural hazard occurs when two instructions require the same hardware resource at the same time. Separate instruction and data caches, or multiple functional units, can help reduce such conflicts.

10. Comparison of Pipeline Depth

Feature	Shallow Pipeline	Deep Pipeline
Number of stages	Low	High
Clock frequency	Usually lower	Potentially higher
Pipeline complexity	Lower	Higher
Branch penalty	Generally smaller	Generally larger
Pipeline registers	Fewer	More
Throughput	Lower	Potentially higher
Control complexity	Lower	Higher

A deeper pipeline can therefore provide higher performance when instructions are independent and branch behavior is predictable. However, hazards and branch mispredictions can reduce its practical advantage.

11. Overall Evaluation

The simulation shows that pipelining improves processor performance mainly by increasing instruction throughput. Arithmetic instructions use the ALU, memory instructions use the memory-access stage, and branch instructions introduce control dependencies.

Increasing pipeline depth can reduce the work performed in each stage and potentially increase clock frequency. However, deeper pipelines require more pipeline registers and have greater penalties when hazards or branch mispredictions occur.

Therefore, there is a trade-off between pipeline depth, clock speed, complexity, and effective performance. The best pipeline depth depends on the processor architecture and the workload being executed.

12. Conclusion

A pipelined processor divides instruction execution into multiple stages so that several instructions can be processed simultaneously. Arithmetic, memory, and branch instructions can all pass through the pipeline, although branch instructions may introduce control hazards.

Pipeline depth has a major influence on execution time, throughput, and speedup. A deeper pipeline can increase clock frequency and throughput, but it also increases complexity and branch penalties. In an ideal situation, a five-stage pipeline can approach one completed instruction per clock cycle after the pipeline is filled. In real processors, data hazards, structural hazards, branch instructions, and other delays reduce the ideal speedup.

Thus, pipeline design involves balancing execution speed, throughput, pipeline depth, hardware complexity, and hazard penalties to achieve the best overall processor performance.

QUESTION – 4

Pipeline with Instruction Dependencies and Branch Conditions

1. Introduction

A pipeline divides instruction execution into several stages so that multiple instructions can be processed simultaneously. A common processor uses a five-stage pipeline consisting of Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB).

Although pipelining improves instruction throughput, it can introduce problems called pipeline hazards. The major hazards are data hazards and control hazards. Data hazards occur when instructions depend on the results of previous instructions, while control hazards occur mainly because of branch instructions.

To overcome these problems, processors use techniques such as stalling, forwarding, and branch prediction.

2. Five-Stage Pipeline

The basic pipeline contains the following stages:

- IF – Instruction Fetch: Fetches the instruction from memory.
- ID – Instruction Decode: Decodes the instruction and reads source registers.
- EX – Execute: Performs an ALU operation or calculates an address.
- MEM – Memory Access: Reads from or writes to memory.
- WB – Write Back: Stores the result into the destination register.

The general flow is:

IF → ID → EX → MEM → WB

When the pipeline is full, different instructions occupy different stages during the same clock cycle.

3. Pipeline with Instruction Dependencies

Consider the following instruction sequence:

I1: ADD R1, R2, R3
I2: SUB R4, R1, R5
I3: AND R6, R4, R7
I4: BEQ R6, R0, TARGET
I5: OR   R8, R9, R10

Here, several dependencies exist:

- I2 depends on I1 because I2 uses R1 produced by I1.
- I3 depends on I2 because I3 uses R4 produced by I2.
- I4 depends on I3 because the branch compares R6.
- I5 may be affected by the branch decision.

These dependencies can cause pipeline delays.

4. Data Hazards

A data hazard occurs when an instruction requires data that is being produced by an earlier

instruction that has not yet completed.

For example:

I1: ADD R1, R2, R3

I2: SUB R4, R1, R5

I2 needs the value of R1 generated by I1. However, I1 may not write R1 until its WB stage. This is called a Read After Write (RAW) hazard.

If the processor does nothing to handle the dependency, I2 could use an old or incorrect value of R1.

5. Stalling Technique

Stalling temporarily stops an instruction from progressing through the pipeline until the required data becomes available.

For example, without forwarding, the pipeline may look like:

Instruction	C1	C2	C3	C4	C5	C6	C7
I1: ADD	IF	ID	EX	MEM	WB		
I2: SUB		IF	ID	STALL	EX	MEM	WB
I3: AND			IF	STALL	ID	EX	MEM

The stall introduces extra clock cycles and therefore increases execution time.

Advantages of Stalling

- Simple to implement.
- Prevents incorrect data from being used.
- Provides reliable execution.

Disadvantages

- Reduces pipeline throughput.
- Increases the number of clock cycles.
- Causes processor resources to remain partially idle.

6. Forwarding Technique

Forwarding, also called data bypassing, reduces stalls by sending the result directly from one pipeline stage to another without waiting for it to be written into the register file.

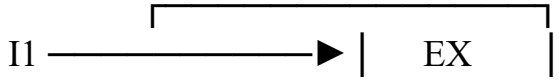
For example:

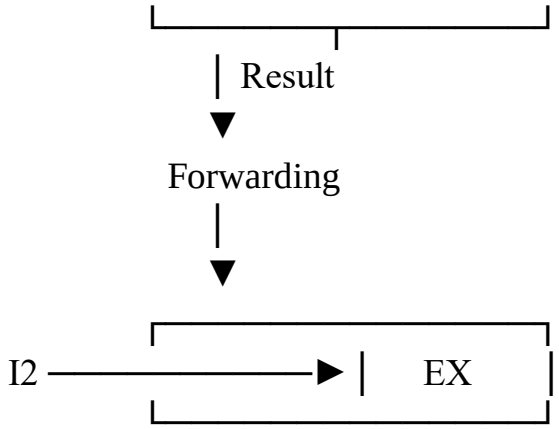
I1: ADD R1, R2, R3

I2: SUB R4, R1, R5

The result generated by I1 in the EX stage can be forwarded directly to the EX stage of I2.

Conceptually:





With forwarding, many data hazards can be resolved without inserting a stall.

Benefits of Forwarding

- Reduces pipeline stalls.
- Improves instruction throughput.
- Decreases execution time.
- Makes better use of processor resources.

However, forwarding cannot solve every hazard. For example, a load-use dependency may still require a stall because loaded data may not become available early enough.

7. Load-Use Hazard

Consider:

I1: LOAD R1, 0(R2)

I2: ADD R3, R1, R4

The LOAD instruction obtains data from memory. The ADD instruction immediately requires that data.

Even with forwarding, the data may not be available early enough for the next instruction's EX stage. Therefore, the processor may need to insert one or more stalls.

A simplified schedule is:

Instruction	C1	C2	C3	C4	C5	C6
LOAD	IF	ID	EX	MEM	WB	
ADD		IF	ID	STALL	EX	WB

This demonstrates that forwarding reduces hazards but does not eliminate all pipeline delays.

8. Control Hazards

A control hazard occurs when the processor encounters a branch instruction and does not immediately know which instruction should be fetched next.

Consider:

I1: ADD R1, R2, R3

I2: BEQ R1, R0, TARGET

I3: SUB R4, R5, R6

I4: AND R7, R8, R9

If the branch condition is true, instructions I3 and I4 may not belong to the correct execution path. They may have to be removed from the pipeline.

This process is called a pipeline flush.

9. Branch Prediction

Branch prediction attempts to predict whether a branch will be taken or not taken before the actual condition is completely evaluated.

There are two basic approaches:

a) Static Branch Prediction

The processor uses a fixed rule.

Examples:

- Always predict branch not taken.
- Always predict branch taken.
- Predict based on the type of branch.

b) Dynamic Branch Prediction

The processor uses information about previous branch behavior to make future predictions.

For example, if a branch was taken repeatedly in previous executions, the processor may predict that it will be taken again.

If the prediction is correct, the pipeline continues normally with little or no interruption.

If the prediction is incorrect, the incorrectly fetched instructions are flushed and the correct instructions are fetched.

10. Performance Impact of Branch Prediction

Suppose a branch is encountered every few instructions.

Without branch prediction, the processor may have to wait until the branch decision is known before fetching the correct next instruction. This creates idle cycles.

With branch prediction:

Correct prediction → No major pipeline interruption

Incorrect prediction → Flush + branch penalty

Therefore, a high prediction accuracy can significantly improve processor performance.

For example, if a five-stage pipeline has a branch penalty of three cycles and branch prediction is highly accurate, only a small number of instructions will suffer the full penalty.

11. Combined Pipeline Example

Consider:

I1: LOAD R1, 0(R2)

I2: ADD R3, R1, R4

I3: SUB R5, R3, R6

I4: BEQ R5, R0, TARGET

I5: AND R7, R8, R9

The dependencies are:

- I2 depends on the result of I1.

- I3 depends on the result of I2.
- I4 depends on the result of I3.
- I5 depends on the branch decision.

A suitable solution is:

1. Use forwarding for the dependencies between I2 and I3.
2. Insert a stall for the load-use dependency between I1 and I2 if necessary.
3. Use branch prediction for I4.
4. If the prediction is incorrect, flush the incorrectly fetched instructions.
5. Continue execution from the correct branch target.

12. Comparison of Hazard-Handling Techniques

Technique	Main Purpose	Performance Impact
Stalling	Wait for required data	Increases execution time
Forwarding	Send results directly between stages	Reduces data stalls
Branch Prediction	Predict branch direction	Reduces control-hazard delays
Pipeline Flush	Remove incorrect instructions	Causes branch penalty
Hazard Detection	Detect dependencies	Prevents incorrect execution

13. Overall Performance Analysis

Pipeline performance is strongly affected by the number and type of hazards.

Without hazard handling, dependencies can cause incorrect results.

With stalling, correct execution is maintained, but additional cycles reduce throughput.

With forwarding, many data hazards are resolved without waiting for register write-back, significantly improving performance.

With branch prediction, the processor can continue fetching instructions before the branch result is known. Correct predictions improve throughput, while incorrect predictions cause pipeline flushing and performance loss.

Therefore, the most effective modern pipeline generally combines hazard detection, forwarding, selective stalling, and branch prediction.

14. Conclusion

A pipelined processor improves performance by executing multiple instructions simultaneously, but instruction dependencies and branches create hazards that can reduce the benefits of pipelining.

Data hazards are mainly handled using forwarding and stalling, while control hazards are reduced using branch prediction. Stalling guarantees correct execution but introduces additional cycles. Forwarding reduces unnecessary waiting, and accurate branch prediction minimizes the penalty caused by branch instructions.

QUESTION – 5

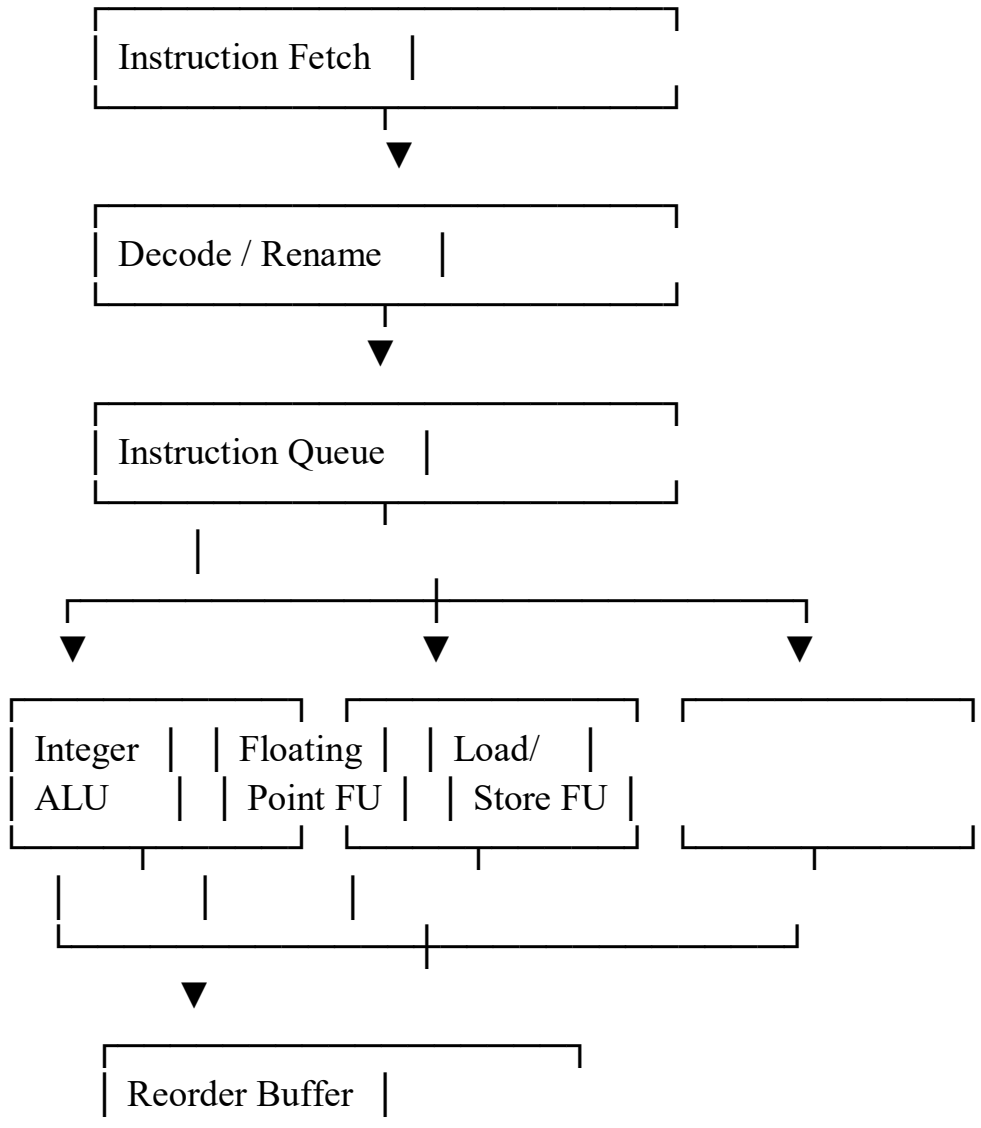
Superscalar Processor with Multiple Execution Units and Out-of-Order Processing

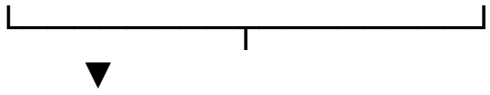
1. Introduction

A superscalar processor is a processor architecture that can execute more than one instruction in a single clock cycle. Unlike a basic scalar processor, which normally processes one instruction at a time, a superscalar processor contains multiple execution units such as arithmetic logic units, floating-point units, load/store units, and branch units. Superscalar processors can also use out-of-order execution, in which instructions are executed when their required operands and execution resources are available rather than strictly following the original program order. This improves processor utilization and increases instruction throughput.

2. Basic Superscalar Processor Model

A simplified superscalar processor can be represented as:





Commit / Write Back

The processor fetches and decodes multiple instructions and places them into an instruction queue. The scheduler then selects instructions that are ready for execution.

3. Multiple Execution Units

A superscalar processor contains several specialized execution units.

a) Integer ALU

The integer ALU performs operations such as:

- Addition
- Subtraction
- AND
- OR
- XOR
- Integer comparisons

b) Floating-Point Unit

The floating-point unit performs operations involving floating-point values, such as floating-point addition and multiplication.

c) Load/Store Unit

The load/store unit handles data transfers between registers and memory.

d) Branch Unit

The branch unit evaluates branch conditions and calculates branch targets.

Having multiple execution units allows independent instructions to execute simultaneously.

For example:

Instruction 1 → Integer ALU

Instruction 2 → Floating-Point Unit

Instruction 3 → Load/Store Unit

These instructions can potentially execute during the same clock cycle.

4. Out-of-Order Instruction Processing

In in-order execution, instructions execute according to their original program sequence.

For example:

I1 → I2 → I3 → I4

If I2 is waiting for memory, I3 and I4 may also have to wait in a simple in-order design.

In out-of-order execution, the processor examines the instructions and executes those whose operands are ready.

For example:

I1: LOAD R1, [Memory]

I2: ADD R2, R3, R4

I3: SUB R5, R6, R7

If I1 is waiting for memory, I2 and I3 do not necessarily have to wait. They can execute using available execution units because they do not depend on I1.

Thus:

I1 → Waiting for memory

I2 → Integer ALU → Execute

I3 → Integer ALU → Execute

This improves utilization of the processor's hardware.

5. Instruction Scheduling

Instruction scheduling determines which instructions should be sent to execution units and when.

The scheduler checks:

- Whether source operands are available.
- Whether the required execution unit is free.
- Whether dependencies exist.
- Whether the instruction has sufficient priority.
- Whether executing the instruction could violate program correctness.

For example:

I1: LOAD R1, [R2]

I2: ADD R3, R4, R5

I3: MUL R6, R7, R8

If the load operation is delayed by memory access, I2 can be assigned to an integer ALU and I3 to a multiplication unit.

This prevents execution units from remaining idle.

6. Register Renaming

Register renaming is an important technique used to eliminate false dependencies between instructions.

There are three major types of dependencies:

- RAW – Read After Write: A true dependency.
- WAR – Write After Read: An anti-dependency.
- WAW – Write After Write: An output dependency.

Register renaming provides temporary physical registers so that independent instructions do not have to use the same architectural register.

For example:

I1: R1 ← R2 + R3

I2: R1 ← R4 + R5

Although both instructions write to R1, they can use different physical registers internally. This allows greater freedom for out-of-order execution.

7. Reorder Buffer

Out-of-order execution creates a problem: instructions may finish in a different order from

the original program.

A Reorder Buffer (ROB) is used to maintain correct program order when results are finally committed.

For example:

Program order:

I1 → I2 → I3 → I4

Execution order:

I1 → I3 → I2 → I4

Even though I3 finishes before I2, the processor can commit the results in the original order:

I1 → I2 → I3 → I4

This ensures correct architectural behavior and supports precise handling of exceptions and interrupts.

8. Example of Superscalar Execution

Consider the following instruction sequence:

I1: LOAD R1, [R2]

I2: ADD R3, R4, R5

I3: MUL R6, R7, R8

I4: SUB R9, R1, R10

I5: AND R11, R12, R13

Assume the processor has:

- One load/store unit.
- One integer ALU.
- One multiplication unit.
- A scheduler capable of out-of-order execution.

A possible execution sequence is:

Cycle	Load/Store Unit	Integer ALU	Multiply Unit
1	I1: LOAD	I2: ADD	I3: MUL
2	—	I5: AND	—
3	—	I4: SUB*	—

The asterisk indicates that I4 must wait for the result of I1 because it has a true data dependency on R1.

While I1 waits for memory, independent instructions I2 and I3 can execute. Therefore, the processor does not remain idle.

9. Resource Allocation

Resource allocation determines which execution unit is assigned to each instruction.

For example:

Instruction Type	Assigned Unit
Integer ADD	Integer ALU
Integer SUB	Integer ALU
Floating ADD	Floating-Point Unit
MUL	Multiply Unit
LOAD	Load/Store Unit
STORE	Load/Store Unit
Branch	Branch Unit

Efficient resource allocation prevents situations in which one execution unit is overloaded while another remains unused.

For example, if two independent instructions require different execution units, they can execute simultaneously. This increases the number of completed instructions per cycle.

10. Effect on Instruction Throughput

Instruction throughput is the number of instructions completed during a given period.

A scalar processor may complete approximately one instruction per cycle under ideal conditions. A superscalar processor with a width of two can potentially issue two instructions per cycle, while a four-wide processor may potentially issue four.

However, the actual throughput is lower than the theoretical maximum because of:

- Data dependencies.
- Branch instructions.
- Cache misses.
- Limited execution resources.
- Instruction scheduling delays.
- Memory latency.

Therefore, increasing the number of execution units alone does not guarantee proportional performance improvement.

11. Performance Improvement Through Scheduling

Instruction scheduling improves throughput by finding independent instructions that can execute while another instruction is waiting.

For example:

I1: LOAD R1, [Memory]

I2: ADD R2, R3, R4

I3: XOR R5, R6, R7

If I1 experiences a memory delay, the scheduler can issue I2 and I3 to available ALUs.

Without out-of-order scheduling:

I1 → wait → I2 → I3

With out-of-order scheduling:

I1 → wait

I2 → execute

I3 → execute

This increases hardware utilization and reduces wasted cycles.

12. Performance Improvement Through Resource Allocation

Resource allocation improves performance by matching instructions with appropriate execution units.

If a processor contains separate ALUs, floating-point units, and load/store units, independent instructions can be processed simultaneously.

For example:

Integer instruction → ALU

Floating instruction → FPU

Memory instruction → Load/Store Unit

This parallel execution increases the number of instructions that can be completed per cycle.

13. Advantages of Superscalar and Out-of-Order Execution

The major advantages include:

1. Higher instruction throughput because multiple instructions can execute simultaneously.
2. Better hardware utilization because idle execution units can be assigned ready instructions.
3. Reduced effect of memory delays because independent instructions can execute while one instruction waits.
4. Improved performance for instruction sequences containing sufficient parallelism.
5. Efficient execution of mixed workloads involving arithmetic, memory, and branch instructions.

14. Limitations

Despite its advantages, superscalar out-of-order execution has several limitations.

- Hardware design is complex.
- Instruction scheduling requires significant control logic.
- Register renaming requires additional hardware.
- Reorder buffers consume processor resources.
- Power consumption can increase.
- Performance is limited when instructions have many dependencies.
- Branch mispredictions can reduce the benefits of parallel execution.

Therefore, superscalar processors require sophisticated hardware to identify and exploit instruction-level parallelism.

15. Overall Evaluation

A superscalar processor improves instruction throughput by combining multiple execution units, instruction scheduling, register renaming, and out-of-order execution.

If instructions are independent, several can execute during the same clock cycle. When one instruction is delayed, the scheduler can select other ready instructions. Resource allocation ensures that instructions are sent to suitable execution units.

For example, if a processor can issue four independent instructions per cycle, its theoretical peak throughput is four instructions per cycle. However, dependencies, branches, memory delays, and resource conflicts reduce the practical throughput.

Thus, performance improvement depends not only on the number of execution units but also on how effectively the processor identifies parallelism and allocates resources.

16. Conclusion

A superscalar processor uses multiple execution units to execute several instructions simultaneously. Out-of-order execution allows ready instructions to execute without unnecessarily waiting for earlier instructions. Instruction scheduling identifies suitable instructions, while resource allocation assigns them to appropriate execution units.

Register renaming removes false dependencies, and the reorder buffer ensures that results are committed in the correct program order. Together, these techniques improve processor utilization and increase instruction throughput.

Therefore, a well-designed superscalar processor can achieve significantly higher performance than a simple scalar processor, especially when the program contains sufficient instruction-level parallelism.